

CRASH Game Specifications

Author: Zhenshuo Zhao

1. Game Overview

1.1. Game Concept

Crash is a real-time multiplayer game in which players place bets before each round begins and attempt to cash out before the multiplier reaches a randomly determined crash point.

At the start of each round, players have a limited betting window to place one or more bets. Once betting closes, the round begins, and the displayed multiplier increases continuously from 1.00x. Players may cash out at any point while the round is active. A successful cash-out pays the player's stake multiplied by the multiplier at the time the cash-out is confirmed by the server.

The round ends when the server-generated crash condition is reached. Players who have not successfully cashed out before the crash receive no payout for that bet.

The game is designed as a real-time multiplayer experience: all players connected to the same round observe the same authoritative game state and multiplier progression, while their individual betting and cash-out actions are processed independently by the server.



1.2. Basic Round Flow

Each round follows a fixed lifecycle:

Betting → Running → Crashed → Settlement → Next Round

During **Betting**, players can submit bets before the server closes the betting window. During **Running**, the multiplier increases continuously and is broadcast to connected clients through the real-time communication layer. Players can request to cash out while the round remains active.

When the **Crashed** condition is reached, the server terminates the round and prevents further successful cash-outs.

During **Settlement**, the system processes the financial results of bets and successful cash-outs before the next round proceeds.

1.3. Technical Characteristics

The game requires the server to maintain authoritative control over:

- Round state and state transitions
- Betting eligibility and acceptance
- Multiplier progression and crash determination
- Cash-out validation
- Player bet and payout records
- Settlement processing
- The leaderboard displaying the top 50 players ranked by bet amount (during **Betting**)
- The leaderboard displaying the top 50 players ranked by cash-out multiplier (during **Running**)

Because the game operates in real time, network latency, delayed messages, duplicate cash-out requests, client disconnections, and settlement processing time must be considered as part of the core game design rather than treated solely as infrastructure concerns.

2. Game States

The game operates through a server-authoritative state machine. The server controls the current round, determines state transitions, and validates player actions based on both the current game state and the player's active bets.

2.1. Round State Definitions

A round progresses through the following states:

BETTING → RUNNING → CRASHED → SETTLEMENT → BETTING

The key difference from a conventional round-based betting model is that betting for the next round can begin while the current round is still in the **RUNNING** state. Once a player has no unresolved active bet in the current round — either because their bet has been successfully cashed out or because they did not place a bet in the current round — they can place a bet for the next round during the current round's **RUNNING** state.

State	Purpose	Player Actions	Server Behaviour	Transition
BETTING	Accept bets for the upcoming round	1. Place bets for the upcoming round	1. Validate account, balance, stake, target round and player eligibility	→ RUNNING
RUNNING	Execute the active round and increase the multiplier	1. Cash out current-round bets 2. Place next-round bets when eligible	1. Maintain authoritative multiplier 2. Process cash-outs 3. Accept eligible next-round bets	→ CRASHED
CRASHED	End active gameplay	1. View final result 2. No further cash-out 3. Place next-round bets when eligible	1. Finalise multiplier 2. Reject late cash-outs 3. Initiate settlement 4. Accept eligible next-round bets	→ SETTLEMENT
SETTLEMENT	Process results from the completed round	1. View settlement status 2. Place next-round bets when eligible	1. Process payouts and unresolved bets 2. Maintain transaction integrity 3. Accept eligible next-round bets	→ BETTING

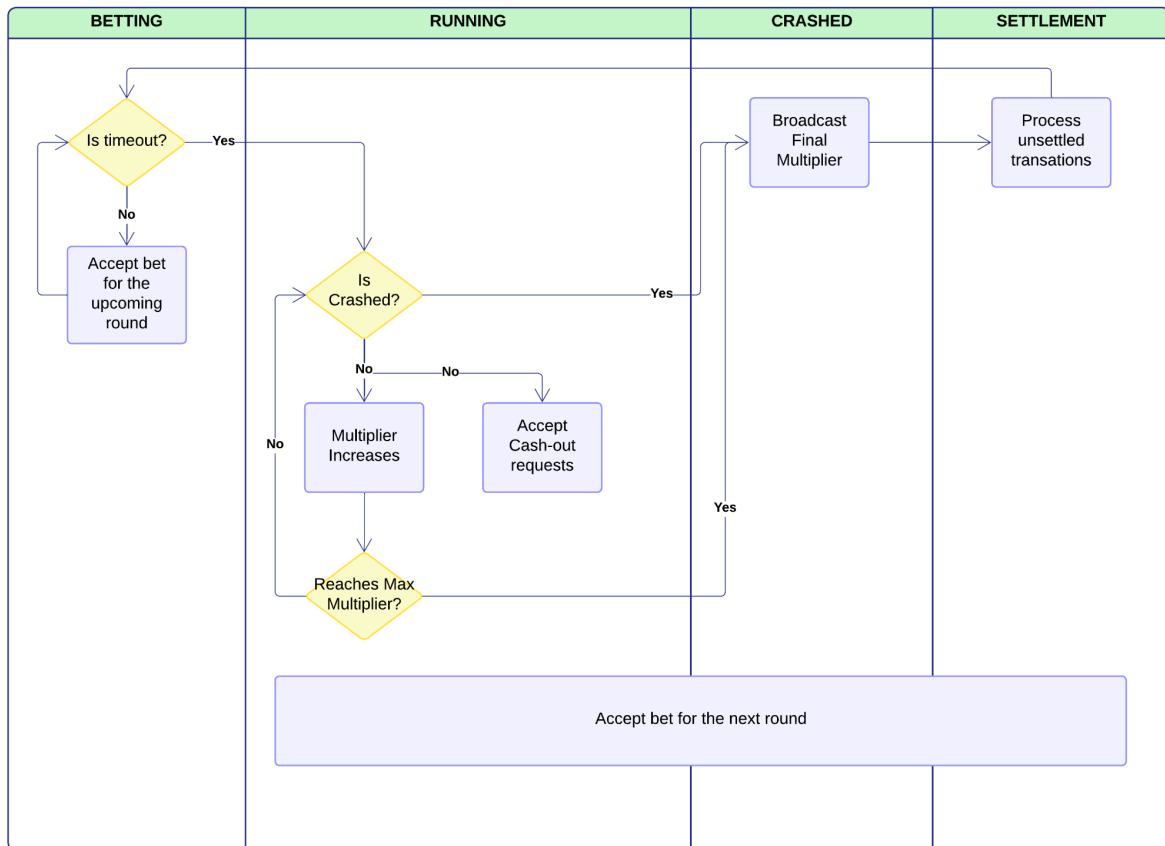
2.2. Player Betting Eligibility

Next-round betting can begin while the current round is still **RUNNING**. Eligibility depends on whether the player has an unresolved active bet in the current round.

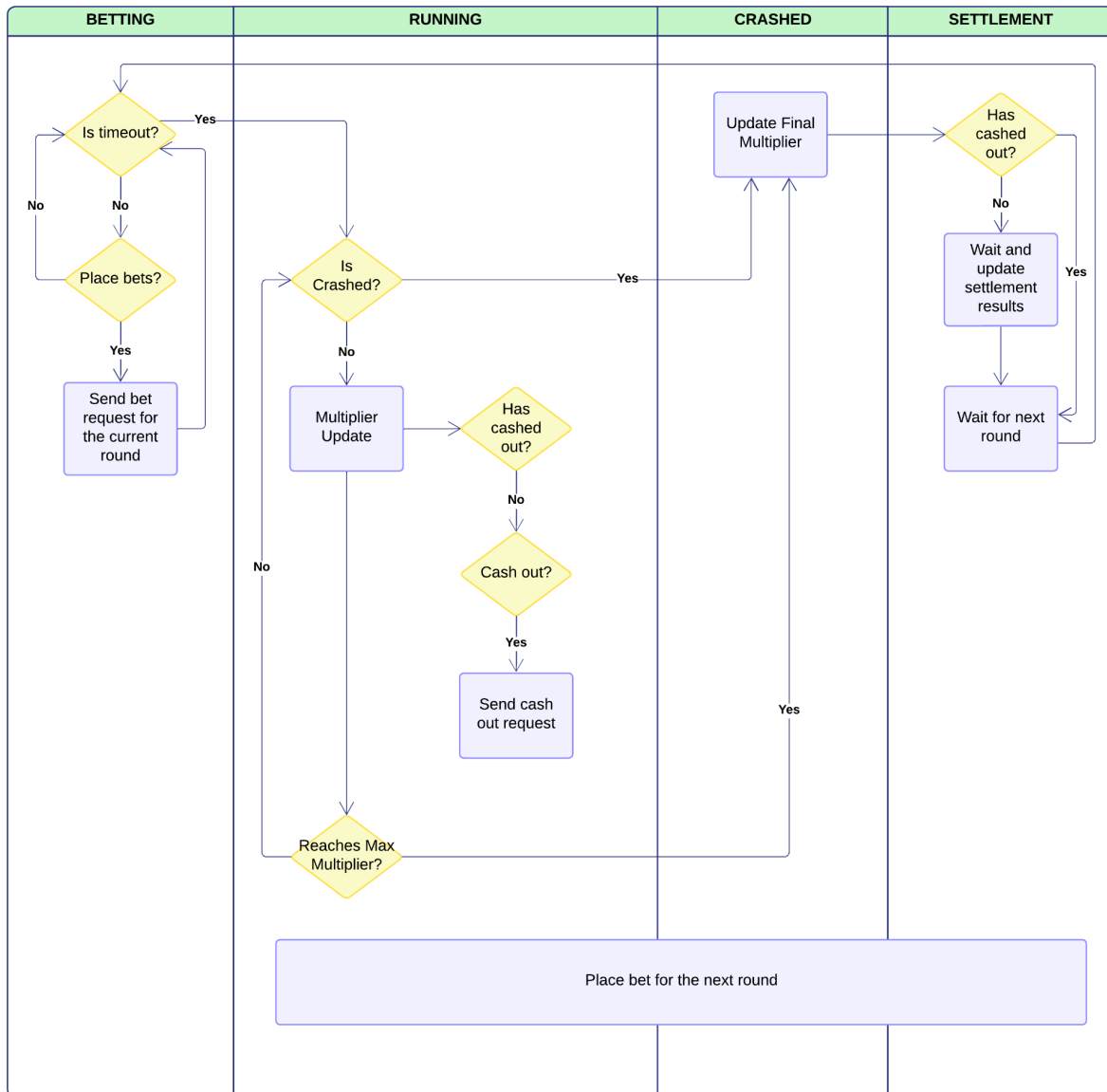
Player Condition	Can Place Next-Round Bet?
No bet placed in current round	Yes
Current-round bet successfully cashed out	Yes
Current-round bet remains active	No
Current round has crashed, and next-round betting remains open	Yes
Next round has entered RUNNING	No

3. Game Flow

Server Flow



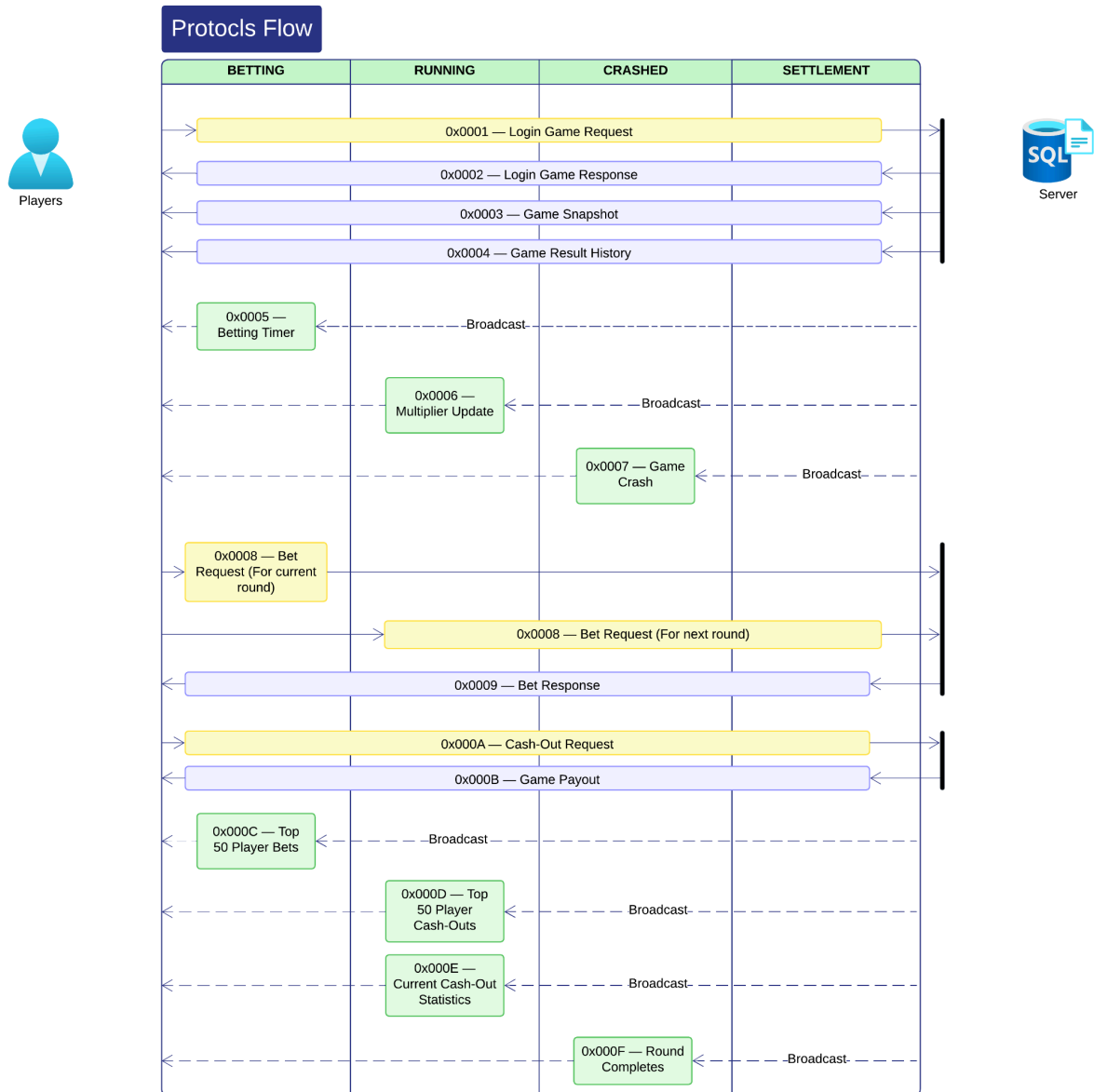
Client Flow



4. WebSocket Protocols

The following protocols assume that the player has already authenticated with the lobby server and obtained the required account information.

The game server uses WebSocket communication to provide real-time game-state synchronisation, player actions, game results, and live statistics.



4.1. Common Data Types

Game Type

gameType (char[4])

The platform uses a four-character game type identifier for all games. The Crash game uses: **CRSH**

Game State

gameState: (uint8)

Value	State
0	Betting
1	Running

2	Crashed
3	Settlement

4.2. Protocol Definitions

0x0001 — Login Game Request

Direction: Client → Server

Trigger: Player enters the Crash game.

Payload

Field	Type	Description
userId	char[10]	Player identifier
gameType	char[4]	Game type identifier (CRSH)

0x0002 — Login Game Response

Direction: Server → Client

Trigger: Response to 0x0001.

Payload

Field	Type	Description
returnCode	uint32	Request result code (0 - succeed, others - error code)
currentRoundBet	double	Player's total accepted bet amount for the current round; 0 if not applicable
nextRoundBet	double	Player's total accepted bet amount for the next round; 0 if not applicable

Note: The bet amounts represent the totals currently accepted by the server.

0x0003 — Game Snapshot

Direction: Server → Client

Trigger: One-shot push immediately after a successful game login.

Provides the client with the current authoritative game state so that the client can initialise its UI without waiting for the next periodic broadcast. It can be used for new logins and reconnections after network issues.

Payload

Field	Type	Description
roundId	uint64	Current round identifier

gameState	uint8	0: Betting, 1: Running, 2: Crashed, 3: Settlement
timeLeft	uint8	Seconds remaining in the betting period; 0 for other states
multiplier	float	Current or final multiplier, depending on game state

Multiplier behaviour

- BETTING: 0
- RUNNING: Current multiplier
- CRASHED: Final crash multiplier
- SETTLEMENT: Final crash multiplier

The multiplier is within the range 0–300 and may contain decimal values.

0x0004 — Game Result History

Direction: Server → Client

Trigger: One-shot push immediately after a successful game login.

Payload

Field	Type	Description
history	float[50]	The latest 50 completed round results. Each element represents the crash multiplier of a completed round.

Notes:

1. The server maintains a cached history queue containing the latest 50 round results.
2. Subsequent updates of the history should be handled by the client, appending the new result from 0x0007.

0x0005 — Betting Timer

Direction: Server → Client

Trigger: Periodic broadcast during the BETTING state.

Payload

Field	Type	Description
gameState	uint8	Always 0 (BETTING)
timeLeft	uint8	Seconds remaining in the betting period

Note: The server broadcasts this message once per second to keep client-side betting timers synchronised with the server.

0x0006 — Multiplier Update

Direction: Server → Client

Trigger: Periodic broadcast during the RUNNING state.

Payload

Field	Type	Description
gameState	uint8	Always 1 (RUNNING)
multiplier	float	Current round multiplier

Note: The server broadcasts the current multiplier once per second to keep clients synchronised with the authoritative game state.

0x0007 — Game Crash

Direction: Server → Client

Trigger: The current round reaches its crash condition.

Payload

Field	Type	Description
gameState	uint8	Always 2 (CRASHED)
multiplier	float	Final multiplier at which the round crashed. When it reaches the maximum 300×, the game completes without a crash.

Note: The server broadcasts the round result to connected clients. Once the game enters CRASHED, further cash-out requests for the round are rejected.

0x0008 — Bet Request

Direction: Client → Server

Trigger: Player submits a bet.

Payload

Field	Type	Description
userId	char[10]	Player identifier
roundId	uint64	Target round identifier
betAmount	double	Amount of the new bet

Notes:

1. A player may submit multiple bets during a round, subject to the game's stake limit.
2. The server validates and persists each accepted bet independently.

0x0009 — Bet Response

Direction: Server → Client

Trigger: Response to each 0x0008 bet request.

Payload

Field	Type	Description
-------	------	-------------

returnCode	uint32	Request result code (0 - succeed, others - error code)
roundId	uint64	Target round identifier
totalBetAmount	double	Total accepted bet amount currently persisted by the server for the player and round

Notes:

1. The totalBetAmount allows the client to synchronise its displayed total with the amount confirmed by the server. For example, if a player submits multiple bets consecutively, the server returns the updated cumulative accepted amount in each corresponding response.
2. All bets within a round should be merged into one order in the database. The order ID corresponds to the payout in 0x000A.

0x000A — Cash-Out Request

Direction: Client → Server

Trigger: Player submits a cash-out request while the round is in the RUNNING state.

Payload

Field	Type	Description
userId	char[10]	Player identifier
timestamp	double	Client timestamp when the cash-out request was submitted

Notes:

1. The server applies a 1-second time tolerance when processing a cash-out request. If the request timestamp falls within the permitted tolerance relative to the authoritative server timing, the cash-out may be accepted.
2. The server remains authoritative for determining whether the cash-out is valid. A request received after the round has entered CRASHED is rejected if it falls outside the permitted timing tolerance.
3. The accepted cash-out timestamp and the corresponding authoritative multiplier are used for subsequent payout processing.

0x000B — Game Payout

Direction: Server → Client

Trigger: Response to each 0x000A cash-out request. Or the player's payout is processed when the game completes without a crash.

Payload

Field	Type	Description
roundId	uint64	Round associated with the payout
payout	double	Payout amount

balance	double	Player account balance update
---------	--------	-------------------------------

Note: This message informs the client of the payout resulting from the completed round.

0x000C — Top 50 Player Bets

Direction: Server → Client

Trigger: Periodic broadcast during the BETTING state.

Payload

Field	Type	Description
stats	stat[50]	The list contains up to 50 players
stat - playerName	char[10]	Display name of a player
stat - betAmount	double	Total bet amount of a player

Note: The server broadcasts the list once per second only when the cached leaderboard data has changed, reducing unnecessary network traffic.

0x000D — Top 50 Player Cash-Outs

Direction: Server → Client

Trigger: Periodic broadcast during the RUNNING state.

Payload

Field	Type	Description
stats	stat[50]	The list contains up to 50 players
stat - playerName	char[10]	Display name of a player
stat - betAmount	double	Bet amount associated with the cash-out
stat - multiplier	float	Cash-out multiplier

Notes:

1. The server broadcasts the list once per second only when the cached leaderboard data has changed.
2. The corresponding payout amount can be calculated on the client-side by (betAmount * multiplier).

0x000E — Current Cash-Out Statistics

Direction: Server → Client

Trigger: Periodic broadcast during the RUNNING state.

Payload

Field	Type	Description
-------	------	-------------

totalPlayers	uint16	Number of players who have placed bets in the current round
currentNumberOfCashedOut	uint16	Number of players who have successfully cashed out

Note: The server broadcasts these statistics once per second during the RUNNING state.

0x000F — Round Completes

Direction: Server → Client

Trigger: Broadcast once when the game switches from the RUNNING state to the SETTLEMENT state.

Payload

Field	Type	Description
gameState	uint8	Always 3 (SETTLEMENT)
roundId	uint64	Round associated with the payout
multiplier	float	The final multiplier when the game completes

5. Transaction & Settlement Logic

The game separates real-time gameplay from financial settlement. The game server remains authoritative for bet acceptance, cash-out validation, round results, and payout calculation, while persistent transaction records provide the source of truth for financial operations.

The settlement flow is designed to maintain balance consistency even when multiple player actions are processed concurrently or when settlement takes longer than the gameplay transition.

5.1. Transaction Lifecycle

A player's financial interaction with a round follows the general lifecycle:

Bet Request → Validation → Bet Persistence → Gameplay → Cash-Out / Crash → Settlement → Payout

Stage	Server Responsibility
Bet Request	Validate the target round, player eligibility, stake amount, and applicable betting rules
Bet Persistence	Persist the accepted bet and update the player's round-level betting total

Gameplay	Maintain the authoritative round state and multiplier
Cash-Out	Validate that the round is still active and record the player's cash-out multiplier
Crash	Finalise the round and reject subsequent cash-out requests
Settlement	Calculate and process the financial outcome of each accepted bet
Payout	Persist the resulting payout and notify the client

5.2. Bet Acceptance

When a player submits a bet, the server validates the request before the bet is considered accepted.

At minimum, validation includes:

1. Confirming that the target roundId is valid.
2. Confirming that the player is eligible to bet on the target round.
3. Confirming that the requested stake is valid.
4. Confirming that the player's available balance is sufficient.
5. Persisting the accepted bet.
6. Updating the player's cumulative accepted bet amount for the round.
7. Returning the server-confirmed total through 0x0009 — Bet Response.

Multiple bet requests may be submitted for the same round. Each accepted request is persisted independently, while the response returns the cumulative amount currently recognised by the server.

5.3. Cash-Out Validation

Cash-out is treated as a time-sensitive server transaction. When a cash-out request is received, the server must validate the request against the authoritative round state rather than relying on the multiplier displayed by the client.

The server determines:

- Whether the target round is still RUNNING
- Whether the player has an active unresolved bet
- The authoritative multiplier at the time the request is accepted
- Whether the bet has already been cashed out or settled

If validation succeeds, the cash-out is recorded against the player's bet and the resulting payout becomes eligible for settlement.

If the round has already entered CRASHED, the cash-out request is rejected, even if the client has not yet received the crash broadcast.

5.4. Crash and Settlement

When the crash condition is reached, the server transitions the round from:

RUNNING → CRASHED → SETTLEMENT

At CRASHED, the final multiplier becomes authoritative, and no additional successful cash-outs can be accepted.

The settlement process then evaluates the accepted bets associated with the completed round.

Bet Outcome	Settlement Result
Game completes without a crash	Calculate and process the corresponding payout for players who haven't cashed out
Game crashes, still active when crash occurred	No game payout
Game crashes, invalid / rejected bet	No settlement
Already settled transaction	No additional payout

5.5. Payout Processing

Payout processing may be handled asynchronously from the real-time game loop. This separation prevents a large settlement workload from blocking the progression of the game itself.

The settlement queue should support retrying failed processing without creating duplicate payouts.

5.6. Settlement and Next-Round Processing

The real-time game loop and settlement workload should be treated as separate processing concerns. For example, if a round generates a large number of payout transactions, the settlement queue may continue processing after the next round has already started.

This allows the game to maintain its real-time operating cadence without requiring every previous-round transaction to finish before the next round begins.

5.7. Failure Handling and Transaction Integrity

Settlement processing must account for failures occurring between transaction steps.

Examples include:

- Database write failure
- Temporary balance-service failure
- Duplicate transaction requests

- Client disconnection
- Settlement worker interruption
- Queue backlog
- Server restart during settlement

The system should use persistent transaction state so that an interrupted settlement can resume safely.

5.8. Consistency Requirements

The settlement architecture should maintain the following invariants:

- A valid accepted bet is associated with exactly one target round. Hence, at most one order per player is generated per round. Multiple bets should be merged into the same order.
- A bet can be successfully cashed out at most once.
- A settlement transaction can result in at most one payout.
- A crashed round cannot accept a successful cash-out after the authoritative crash transition.
- Settlement for one round cannot modify the result of another round.
- Client reconnects must not create duplicate financial transactions.
- Retrying a failed settlement must be safe and idempotent.

6. Edge Cases & Failure Handling

Because the game processes real-time player actions and financial transactions, the server must handle delayed requests, disconnections, duplicate messages, and unexpected failures without compromising game or transaction integrity.

6.1. Edge Cases

Scenario	Expected Handling
Late bet request	Reject if the target round is no longer accepting bets
Late cash-out request	Apply the defined timing tolerance if the game is still running; otherwise reject
Multiple cash-out requests	Only the first valid request in the queue is accepted if the game state allows
Player disconnects	Preserve the server-side bet and settlement state
Server crash	Recover unsettled transactions after restart
Settlement backlog	Process transactions asynchronously without blocking the next round

Client/server state mismatch or the client reconnects	Server state remains authoritative; client resynchronises using the game snapshot
---	---

6.2. Settlement Recovery

Settlement processing must be recoverable after an unexpected server failure. All transactions that have not reached a final settlement state remain persisted as unsettled orders. When the server resumes, it identifies these orders and spawns settlement workers to process them.

Workers must be safe to retry. A transaction that has already been successfully settled must not generate a duplicate payout.

Manual settlement tools should also be provided to the operations team to review and process unsettled transactions that cannot be automatically handled after server recovery.

7. Operator-Specific Stake Configuration

Each game operator is assigned a unique PID (Partner ID). The game supports operator-specific stake range configuration, allowing operations teams to define the minimum and maximum stake amounts permitted for players under each PID.

The server validates the configured stake range when processing each bet request and rejects bets outside the permitted limits.